

TravelIntel AI

Complete Beginner-Friendly System Guide

How the website works, what every file does, and how the pieces fit together

Includes: full architecture walkthrough, file-by-file explanation,
and a 9-part group presentation breakdown (with suggested order)

Table of Contents

- 1. What is TravelIntel AI? (Plain-English Overview)
- 2. The Big Picture: How the System is Built
- 3. File-by-File Explanation
- 4. How the Files Talk to Each Other (Request Flow)
- 5. Key Coding Concepts Explained Simply
- 6. The Database Explained
- 7. The 9 Presentation Parts (with suggested order)
- 8. Tips for Presenting Like a Pro

1. What is TravelIntel AI?

TravelIntel AI is a **travel package booking website**. Think of it like a mini version of Booking.com or Expedia, but built by your group as a learning project. It lets two types of people use it:

- **Customers** — regular visitors who browse travel packages (e.g. "7 days in Cape Town"), create an account, book a trip, and manage their profile and bookings.
- **Admins** — the staff running the business. They log into a separate admin dashboard to manage packages, see bookings, read AI-generated insights, manage reviews, and generate reports.

On top of the normal booking website, the system also has a bit of **Artificial Intelligence** built in: it can predict future booking demand, group customers into segments (e.g. "big spenders" vs "occasional bookers"), and flag unusual/suspicious activity — all using simple, well-known machine learning techniques.

The three simple jobs of the system

- **Show information** — display travel packages, prices, and reviews to visitors.
- **Store information** — save accounts, bookings and reviews in a database so nothing is lost.
- **Think about information** — use AI/statistics to turn raw bookings into useful business insights for admins.

2. The Big Picture: How the System is Built

The system follows a very common web-development pattern called **client-server**. In simple terms:

- Your **web browser** (Chrome, Safari, etc.) is the **client** — it shows pages and sends clicks/forms to the server.
- A **Python program** running on a computer is the **server** — it receives those requests, does the work (checking passwords, saving bookings, running AI), and sends back an answer (usually a web page or a small piece of data called JSON).
- A **database file** stores everything permanently, like a very organized spreadsheet that the server reads from and writes to.

The technology stack (the tools used)

- **FastAPI** — a Python framework (a ready-made toolkit) used to build the server. It listens for web requests such as "show me the packages page" or "log this user in".
- **Jinja2** — a template engine that lets Python insert live data (like a list of packages) into HTML pages before sending them to the browser.
- **SQLite** — a lightweight, file-based database (the whole database is just one file: `travelintel.db`). Perfect for small/medium projects because it needs no separate database server.
- **scikit-learn / pandas / numpy** — Python libraries used for the AI features (forecasting, clustering, anomaly detection).

- **HTML, CSS (Tailwind-style utility classes) and JavaScript** — used in the browser to display pages and make them interactive (buttons, forms, live updates without reloading the page).

A simple diagram in words

Browser (HTML/CSS/JS) ↔ FastAPI server (app.py) ↔ Python helper files (database.py, ai_engine.py, google_reviews_sync.py) ↔ SQLite database file (travelintel.db)

Every click on the website eventually turns into a request that travels along this chain and a response that travels back.

3. File-by-File Explanation

Below is every file in the project and what it is responsible for, explained simply.

File / Folder	Role in the system
<code>app.py</code>	The heart of the system (the 'server'). This one file defines every web page and every API endpoint — login, register, book a trip, admin dashboard data, reports, reviews, AI results, etc. It receives requests from the browser and decides what to do.
<code>config.py</code>	A small settings file. Holds constants like session timeout length, max login attempts before lockout, and the admin recovery email — values that control app behaviour without digging through <code>app.py</code> .
<code>database.py</code>	Everything to do with the database lives here: creating tables the first time the app runs, opening a database connection, and helper functions like getting/saving review summaries and backing up the database.
<code>ai_engine.py</code>	The 'brain' of the system. Contains three AI/analytics functions: demand forecasting (predicts future bookings), customer segmentation (groups similar customers), and anomaly detection (flags unusual booking activity).
<code>google_reviews_sync.py</code>	Connects to Google (via the SerpAPI service) to fetch real Google Reviews for the business, and calculates a simple sentiment score (positive/neutral/negative) for each review.
<code>.env</code>	A private file holding secret configuration values (API keys, IDs) that the app reads at startup. It is kept separate from the code so secrets are never hard-coded.
<code>requirements.txt</code>	The shopping list of Python packages the project depends on (FastAPI, pandas, scikit-learn, etc.). Running <code>pip install -r requirements.txt</code> installs them all.
<code>instance/travelintel.db</code>	The actual SQLite database file — the permanent storage for every user, package, booking and review. This is auto-created/updated by <code>database.py</code> .
<code>templates/</code>	Folder of HTML files. Each file is a page the browser can show (home page, login page, booking page, admin pages, etc.). Jinja2 fills them with live data from the server before sending them out.
<code>templates/landing.html</code>	The very first marketing/welcome page a new visitor sees, before entering the main site.
<code>templates/index.html</code>	The main customer-facing page: browse and search travel packages.
<code>templates/auth.html</code>	Combined login and sign-up page for customers.

File / Folder	Role in the system
templates/forgot-password.html	Lets a customer reset a forgotten password using a one-time code (OTP) sent by email.
templates/booking.html	The page where a customer fills in traveler details and confirms a booking for a package.
templates/confirmation.html	A simple 'thank you, your booking is confirmed' page shown after a successful booking.
templates/profile.html	Lets a logged-in customer view/edit their profile and see their past bookings.
templates/admin/login.html	Separate login page just for admin/staff accounts.
templates/admin/forgot-password.html	Password recovery flow for the admin account.
templates/admin/dashboard.html	The admin's home screen: key numbers at a glance (bookings, revenue, etc.).
templates/admin/packages.html	Where admins add, edit prices, and set the number of available spots for each travel package.
templates/admin/reviews.html	Where admins view customer/Google reviews and manage the review summary shown to customers.
templates/admin/ai-analysis.html	Displays the results of the AI engine — forecasts, customer segments, and anomaly alerts — in a readable way.
templates/admin/reports.html	Lets admins generate and download business reports for a chosen date range.
static/js/main.js	JavaScript that powers the customer-facing pages — loading packages from the server and updating the page without a full reload.
static/js/auth.js	JavaScript for the login/sign-up page — sending the form data to the server and showing errors/success messages.
static/js/package-images.js	A small helper that matches each travel package to a nice-looking cover image.
COMPLETE_FREE_SETUP_GUIDE.md / SETUP_GOOGLE_REVIEWS.md	Plain-text setup instructions for getting the project (and the Google Reviews integration) running on a new computer. Useful reference notes, not code the app needs to run.

4. How the Files Talk to Each Other (Request Flow)

It helps to trace one real action through the whole system. Let's follow: **"A customer books a trip to Cape Town."**

- 1 The customer is on `booking.html` in their browser and clicks **Confirm Booking**.
- 2 JavaScript on that page collects the form data (name, email, dates, number of travelers) and sends it to the server as a request to `POST /bookings`.
- 3 `app.py` receives this request. It checks the customer is logged in, checks the package has enough available spots, and validates the data using a Pydantic model (a Python class that automatically checks that the incoming data has the right shape and types).
- 4 `app.py` calls into `database.py` to open a connection to `travelintel.db`, insert a new row into the Bookings table, and reduce the package's `available_spots`.
- 5 `app.py` sends a confirmation email to the customer (using the email-sending helper functions near the top of the file).
- 6 The server sends back a small JSON response like `{"success": true}`.
- 7 The browser's JavaScript sees the success response and redirects the customer to `confirmation.html`.

Every other feature (login, admin editing a package, generating a report, viewing AI analysis) follows this exact same pattern: **Browser asks** → **app.py decides** → **database.py / ai_engine.py / google_reviews_sync.py do the work** → **app.py replies** → **Browser updates the page**.

5. Key Coding Concepts Explained Simply

API Endpoint

A specific web address the server listens on, e.g. `/api/packages`. Each endpoint is just a Python function marked with a decorator like `@app.get("/api/packages")`, which tells FastAPI 'run this function whenever someone visits this address'.

GET vs POST vs PUT vs DELETE

These are the four common 'verbs' of the web. GET = fetch/read data (e.g. view packages). POST = create something new (e.g. register, book a trip). PUT = update something that already exists (e.g. edit a package's price). DELETE = remove something (e.g. delete a review).

Session / Cookie

After logging in, the server can't 'remember' the user between page visits by itself — the web is stateless. So the server creates a **session** (a temporary record of who is logged in) and gives the browser a small ID called a **cookie**. The browser automatically sends that cookie back on every future request, so the server can recognise the user.

Password Hashing

Passwords are never stored as plain text. `werkzeug.security` is used to turn a password into a scrambled ('hashed') value that can be checked but not reversed. Even if someone stole the database, they could not read the actual passwords.

Pydantic Model

A Python class (e.g. `BookingRequest`) that describes exactly what fields a piece of incoming data must have and what type they should be. FastAPI uses it to automatically reject bad/incomplete requests before your code even runs.

Template Rendering (Jinja2)

HTML files in the `templates/` folder can contain special placeholders. The server 'renders' the template, replacing placeholders with real data, before sending finished HTML to the browser.

JSON

A simple text format for sending structured data (like a list of packages) between the server and the browser's JavaScript. It looks like Python dictionaries and lists.

SQL / Database Queries

SQL is the language used to talk to the database — e.g. `SELECT * FROM Packages` means 'give me every row from the Packages table'. `database.py` and parts of `app.py` and `ai_engine.py` build and run these queries.

Machine Learning (the AI part)

The AI engine uses simple, well-established techniques: **Linear Regression** to draw a trend line through past bookings and predict future demand; **K-Means Clustering** to automatically group customers with similar booking behaviour; and basic statistics to detect bookings that look unusual compared to the normal

pattern.

Background Scheduler

The APScheduler library runs certain jobs automatically in the background on a timer (e.g. periodically re-training the AI models or syncing Google Reviews) without anyone needing to click a button.

6. The Database Explained

The whole database lives in one file, `instance/travelintel.db`, and is made up of tables (like tabs in a spreadsheet). `database.py` creates these automatically the first time the app runs. Here are the main ones, in plain English:

Table	What it stores
Users	Admin/staff accounts — username, hashed password, and role.
Customers	Everyday visitor accounts — email, hashed password, profile details.
Packages	Every travel package on offer — name, destination, price, description, total and available spots, status.
Bookings	Every trip booked by a customer — which package, traveler details, number of travelers, payment method, date.
Reviews	Customer/Google reviews left about the business, with rating and text.
Review_Summary	A quick-access summary (average rating, total review count) so the site doesn't have to recalculate it on every page load.
Analytics_Log	A record of analytics/AI events, used for tracking and auditing.
Alerts	Anomaly-detection warnings raised by the AI engine (e.g. an unusually large booking).
Forecasts	Saved output from the demand-forecasting AI model.
Reports	Generated business reports (so admins can re-download a past report instead of regenerating it).
App_Settings	Miscellaneous configuration values the app can change while running.
Sessions	Active logged-in sessions, linking a browser cookie to a specific user.

Think of **Packages** and **Bookings** as the two most important tables — everything else exists to support, secure, analyse, or report on the data inside them.

7. The 9 Presentation Parts

The system has been divided into **9 parts** so each group member can own one coherent slice of the system. The parts are ordered so the presentation tells a logical story: start with the big picture, then follow a visitor's journey through the site (browse → sign up → book), then move into the admin side, and finish with the 'smart'/automated features.

PART 1

Project Overview & System Setup

Files/areas covered: `config.py`, `requirements.txt`, `.env`, `COMPLETE_FREE_SETUP_GUIDE.md`, `SETUP_GOOGLE_REVIEWS.md`

What this part is about

The 'big picture' introduction to the whole project — what TravelIntel AI is, the technology stack used, and how the project is configured and set up.

What it does

`config.py` centralises app-wide settings (session timeout, login lockout rules, admin recovery email). `requirements.txt` lists every external Python library the project needs. `.env` securely stores secret keys. The setup guides document how to install and run the project from scratch.

What it adds to the system

Gives the audience the vocabulary and mental model needed to understand every part that follows — this is the 'table of contents' of the presentation.

How it connects to other parts

Every other part depends on the settings and packages defined here; it is the foundation everything else is built on.

PART 2

The Database Layer

Files/areas covered: `database.py`, `instance/travelintel.db`

What this part is about

How and where the system permanently stores its data.

What it does

`database.py` defines every table (Users, Customers, Packages, Bookings, Reviews, etc.), creates them automatically on first run, opens/closes database connections safely, and provides helper functions (e.g. reading/writing the review summary, backing up the database).

What it adds to the system

Without this layer nothing could be remembered — every booking, account and review would disappear the moment the server restarted.

How it connects to other parts

Almost every other part reads from or writes to this layer: bookings (Part 5), admin package management (Part 6), AI analysis (Part 8) and reports (Part 9) all pull their data from here.

PART 3

Customer Landing & Package Browsing (Frontend)

Files/areas covered: templates/landing.html, templates/index.html, static/js/main.js, static/js/package-images.js

What this part is about

The very first thing a visitor sees, and the main page where they browse available travel packages.

What it does

landing.html is the welcome/marketing page. index.html displays the list of packages, pulling live data from the server. main.js fetches package data from the API and updates the page without reloading it. package-images.js matches each package to a fitting photo.

What it adds to the system

This is the shop window of the whole business — first impressions and the core browsing experience that leads a visitor towards booking.

How it connects to other parts

The package data shown here comes from the API endpoints defined in app.py (Part 5), which in turn comes from the database (Part 2). Clicking a package leads into the booking flow (Part 5).

PART 4

User Accounts & Authentication

Files/areas covered: templates/auth.html, templates/forgot-password.html, templates/profile.html, static/js/auth.js

What this part is about

How visitors create an account, log in, recover a forgotten password, and manage their profile.

What it does

auth.html provides combined sign-up/login forms; auth.js sends that form data to the server and shows success/error feedback. forgot-password.html drives the one-time-code (OTP) password reset flow.

profile.html lets a logged-in customer view/edit their details and see past bookings.

What it adds to the system

Security and personalisation — without accounts, there would be no way to know who is booking a trip or protect customer data.

How it connects to other parts

The actual login/registration logic lives in app.py's auth endpoints (checked together with Part 5), which check credentials against the Customers table (Part 2) and create a session (cookie) that every later request relies on, including booking (Part 5).

PART 5

The Booking System

Files/areas covered: templates/booking.html, templates/confirmation.html, app.py – booking & auth routes (register, login, /bookings, /api/my-bookings, /api/packages)

What this part is about

The core transaction of the whole website: turning a browsing customer into a paying traveler.

What it does

booking.html collects traveler details and payment method and shows live availability. app.py's booking-related routes validate the request, reduce the package's available spots, save the booking to the database, and send a confirmation email. confirmation.html shows the 'success' screen afterwards.

What it adds to the system

This is the revenue-generating heart of the system — everything before it (browsing, accounts) exists to lead here, and everything after it (admin, AI, reports) exists to manage what happens as a result.

How it connects to other parts

Reads package availability from the database (Part 2), requires a logged-in session from Part 4, and every booking created here becomes the raw data used by the admin dashboard (Part 6), AI engine (Part 8) and reports (Part 9).

PART 6

Admin Dashboard & Package Management

Files/areas covered: templates/admin/login.html, templates/admin/forgot-password.html, templates/admin/dashboard.html, templates/admin/packages.html, app.py – admin login & package management routes

What this part is about

The separate, secured area where staff manage the business day-to-day.

What it does

Admin login/forgot-password pages authenticate staff separately from customers, with login-attempt limits for security. `dashboard.html` shows key numbers at a glance. `packages.html` lets admins add packages, edit prices, and set the number of available spots (which customers immediately see reflected on the booking page).

What it adds to the system

Turns the system from a static website into a manageable business tool that staff can run without touching code.

How it connects to other parts

Directly edits the same Packages table that Part 3 (browsing) and Part 5 (booking) read from — changes here are instantly visible to customers.

PART 7

Reviews & Google Integration

Files/areas covered: `google_reviews_sync.py`, `templates/admin/reviews.html`, `app.py` — `/api/reviews` routes

What this part is about

How the system collects and manages customer trust signals — reviews and ratings.

What it does

`google_reviews_sync.py` connects to Google (via the SerpAPI service) to pull in real Google Reviews and calculates a simple sentiment score for each one. The admin reviews page lets staff view, add, edit, delete reviews and refresh the review summary shown to customers.

What it adds to the system

Builds credibility with new customers by showing real social proof, and gives admins visibility into customer sentiment without manually checking Google.

How it connects to other parts

Saves results into the Reviews and Review_Summary tables (Part 2); the average rating pulled here can also be displayed back on the customer-facing pages (Part 3).

PART 8

The AI & Analytics Engine

Files/areas covered: `ai_engine.py`, `templates/admin/ai-analysis.html`, `app.py` — `/api/admin/ai-analysis` & `dashboard-stats` routes

What this part is about

The 'smart' part of the system that turns raw booking history into business intelligence.

What it does

ai_engine.py trains a Linear Regression model to forecast future booking demand from the last 30 days of bookings, uses K-Means Clustering to automatically group customers into segments, and runs simple anomaly detection to flag unusual booking activity. ai-analysis.html presents all of this to admins in a readable dashboard.

What it adds to the system

Moves the business from just reacting to bookings towards predicting demand and understanding customers — the key 'AI' selling point of the whole project.

How it connects to other parts

Reads its raw material entirely from the Bookings table (Part 2), and its output (forecasts, segments, alerts) feeds directly into the admin dashboard (Part 6) and the reports generated in Part 9.

PART 9

Reports & Automated Notifications

Files/areas covered: templates/admin/reports.html, app.py — /api/reports routes and email-sending functions

What this part is about

How the system summarises everything that happened over a period, and how it communicates automatically with customers and admins by email.

What it does

The reports page lets an admin pick a date range and generate a business report (bookings, revenue, trends) that is saved so it can be re-downloaded later. Separately, app.py contains email-sending functions that automatically send account-verification codes, password-reset codes, and booking confirmation emails.

What it adds to the system

Closes the loop — it's how the business reviews its own performance over time, and how the system keeps humans (customers and admins) informed without manual effort.

How it connects to other parts

Reports are built from the same booking, package and AI data produced in every earlier part; it is a natural closing chapter that summarises the whole system's output.

8. Tips for Presenting Like a Pro

- **Start with the story, not the code.** Explain what your part lets a real person (customer or admin) actually do, before showing any file.
- **Show, don't just tell.** If possible, click through the actual feature in the running website before opening the code.
- **Use the flow from Section 4** as a template: 'the user does X → the browser sends a request → this file handles it → the database is updated → the user sees Y'.
- **Name-drop the concepts from Section 5** confidently (e.g. "we hash passwords so they're never stored in plain text") — it shows understanding, not memorisation.
- **Always mention the hand-off.** End your part by pointing to the next presenter: "...and that booking data is exactly what Part 8's AI engine uses next."
- **Keep code snippets short.** Show 5-10 relevant lines, not a whole file — explain what they do rather than reading them aloud.
- **Rehearse the order** in Section 7 as a group beforehand so transitions between presenters feel natural.

Good luck with your presentation!